

Portafolio DevOps

Sistema Hospitalario DevOps

Estudiante: Vania Guarachi Ramos

Repositorio principal: <https://github.com/VaniaGuarachi/SistemaHospitalario-DevOps>

Fork de trabajo: <https://github.com/guarachiramosv/SistemaHospitalario-DevOps>

Pull Request: <https://github.com/guarachiramosv/SistemaHospitalario-DevOps/pull/1>

Actividad: Producto integrador de autoaprendizaje sobre fundamentos DevOps.

1. Certificado de finalizacion

La siguiente captura evidencia el avance en Microsoft Learn dentro de la ruta Fundamentos de DevOps: Principios y practicas fundamentales.



2. Resumen tecnico

Resumen tecnico: Fundamentos de DevOps

1. Que es DevOps y cuales son sus beneficios

DevOps es una forma de trabajar que busca unir a los equipos de desarrollo y de operaciones, que tradicionalmente funcionan separados y con objetivos distintos: el equipo de desarrollo quiere sacar funciones nuevas rapido, mientras que el equipo de operaciones prioriza la estabilidad del sistema. Cuando estos dos equipos no colaboran, aparecen retrasos en los lanzamientos y mas fallas en produccion. DevOps ataca ese problema combinando personas, procesos y herramientas para que ambos equipos trabajen de forma coordinada durante todo el ciclo de vida de la aplicacion, desde la planificacion hasta la operacion.

No es una herramienta ni una tecnologia puntual, sino una cultura que se sostiene en la colaboracion, la transparencia, la responsabilidad compartida y el aprendizaje continuo. Entre sus principales beneficios estan: entregar software mas rapido al mercado, adaptarse mejor a los cambios y a la competencia, mantener sistemas mas estables y confiables, y reducir el tiempo que toma recuperarse cuando algo falla, lo que se conoce como MTTR o tiempo medio de recuperacion. En resumen, DevOps ayuda a que la empresa entregue valor a sus usuarios de forma mas agil y con menos friccion entre equipos.

2. Planificacion agil con GitHub Projects

La planificacion agil se apoya en marcos de trabajo como Scrum y Kanban. Scrum organiza el trabajo en sprints, que son ciclos cortos de una a cuatro semanas, con roles definidos como Product Owner, Scrum Master y equipo de desarrollo. Tambien usa ceremonias como el daily standup, la planificacion de sprint, la revision y la retrospectiva. Kanban, en cambio, funciona con un flujo continuo de trabajo visualizado en un tablero con columnas, por ejemplo: por hacer, en curso y hecho, sin sprints fijos, y suele limitar la cantidad de tareas que se pueden trabajar a la vez mediante limites de WIP.

GitHub Projects es la herramienta que permite aplicar estos marcos directamente sobre el repositorio: organiza el trabajo en tableros tipo Kanban, se integra con los issues para reportar tareas o errores y con los pull requests, y permite automatizar cambios de estado, por ejemplo, mover una tarjeta a listo automaticamente cuando se cierra un issue o se combina un pull request. Esto hace que la planificacion y el seguimiento del proyecto queden en el mismo lugar donde se desarrolla el codigo, lo que facilita la comunicacion entre los miembros del equipo.

3. Control de versiones con Git y GitHub

Git es un sistema de control de versiones distribuido: cada desarrollador tiene una copia completa del repositorio en su computadora, con todo el historial de cambios, y puede trabajar sin depender todo el tiempo de una conexion a un servidor central. Esto contrasta con los sistemas centralizados, donde existe una unica copia del repositorio y hay que estar conectado a ella para casi cualquier operacion.

Los conceptos clave de Git incluyen el commit, que es una foto de los cambios hechos en un momento dado; la rama o branch, que es una linea de desarrollo independiente que permite trabajar en una funcion sin afectar el codigo principal; y el merge, que es la accion de unir los cambios de una rama con otra. GitHub anade sobre Git una capa de colaboracion: aloja los repositorios en la nube, permite crear pull requests para proponer y revisar cambios antes de integrarlos a la rama principal, y ofrece funciones como issues, proteccion de ramas y flujos de trabajo automatizados.

El flujo tipico de trabajo, conocido como GitHub Flow, consiste en crear una rama a partir de la rama principal, hacer los cambios y confirmarlos con mensajes descriptivos, abrir un pull request para que el equipo lo revise, atender los comentarios de la revision y, una vez aprobado, fusionar los cambios a la rama principal. Este proceso es fundamental en DevOps porque permite que varias personas trabajen en paralelo sin pisarse el trabajo, mantiene un historial claro de por que se hizo cada cambio y sirve de base para automatizar pruebas y despliegues.

4. Integracion y entrega continua CI/CD

La integracion continua, o CI, consiste en compilar y probar el codigo automaticamente cada vez que alguien sube un cambio al repositorio. La idea es detectar errores lo antes posible, cuando todavia son baratos y rapidos de corregir, en lugar de descubrirlos semanas despues. La entrega continua, o CD, toma esos artefactos ya probados y automatiza su paso por distintos entornos como pruebas, staging y produccion, dejandolos listos para ser publicados. Cuando ademas se automatiza por completo el paso a produccion sin aprobacion manual, se habla de

despliegue continuo.

En GitHub, esto se implementa con GitHub Actions: se crea un archivo YAML dentro de la carpeta `.github/workflows` que define un flujo de trabajo o workflow. Ese flujo se dispara con un evento, por ejemplo un push, y esta compuesto por jobs, que a su vez contienen steps o pasos individuales, como descargar el código, instalar dependencias, compilar o correr pruebas. Gracias a esta automatización, cada cambio que llega al repositorio pasa por el mismo proceso de verificación, lo que reduce errores humanos y acelera la entrega de nuevas versiones a los usuarios.

5. Observabilidad y excelencia operativa

La observabilidad es la capacidad de entender que está pasando dentro de un sistema a partir de los datos que este genera hacia afuera: registros o logs, métricas y trazas. No se trata solo de saber si algo falló, sino de poder entender por qué falló y dónde, sin necesidad de modificar el código para investigarlo. Esto se logra con supervisión continua, que recolecta esta información en tiempo real y genera alertas cuando algo se sale de lo normal, permitiendo reaccionar rápido antes de que el problema afecte a más usuarios.

La excelencia operativa es el objetivo hacia el que apunta todo esto: mantener los sistemas estables, confiables y con el menor tiempo de inactividad posible. Prácticas como las operaciones continuas, la seguridad continua y el uso de presupuestos de error ayudan a que los equipos encuentren un equilibrio razonable entre confiabilidad, costo y ritmo de cambio, en lugar de perseguir una confiabilidad perfecta a cualquier precio.

3. Repositorio en GitHub

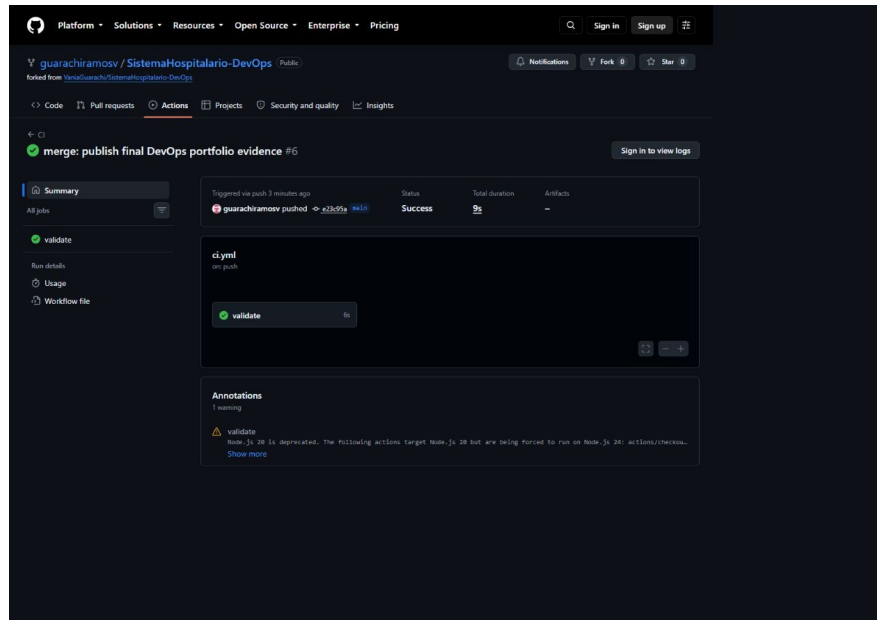
Evidencia solicitada	Estado en el proyecto
Repositorio SistemaHospitalario-DevOps	https://github.com/VaniaGuarachi/SistemaHospitalario-DevOps
README.md descriptivo	Incluido con objetivo, alcance, estructura, ramas, pipeline y enlaces de evidencia.
Al menos 5 commits	Historial con commits descriptivos para documentacion, evidencia, login, CI, PDF y merge.
Rama develop	Creada, publicada y usada como rama de integracion.
Rama feature/login	Creada y publicada para el modulo de inicio de sesion.
Pull Request fusionado a develop	PR #1 fusionado en fork: https://github.com/guarachiramosv/SistemaHospitalario-DevOps/pull/1

4. Flujo CI/CD con GitHub Actions

El archivo `.github/workflows/ci.yml` ejecuta un job llamado `validate` en Ubuntu. Primero descarga el repositorio, luego configura Python 3.12 y finalmente ejecuta `python -m unittest discover -s tests`.

La prueba valida que existan los archivos principales del proyecto y que `login.html` incluya usuario, contraseña, campo password, boton submit y texto de ingreso. No se despliega una aplicacion porque la actividad solo exige una prueba basica o compilacion sencilla.

Ejecucion exitosa: <https://github.com/guarachiramosv/SistemaHospitalario-DevOps/actions/runs/34673776804>



5. Reflexion critica

Diferencia entre Agile y DevOps

Agile organiza el desarrollo en ciclos cortos y entregas incrementales. DevOps conecta ese desarrollo con operaciones, seguridad, pruebas, automatizacion y monitoreo para entregar y operar mejor el producto.

Por que Git es fundamental en DevOps

Git permite controlar versiones, crear ramas, revisar cambios y recuperar estados anteriores. Es la base para automatizar pruebas, auditar cambios y coordinar equipos.

Que problema resuelve la Integracion Continua

La CI evita descubrir errores demasiado tarde. Cada push puede ejecutar pruebas automaticas, detectar fallos rapido y proteger las ramas principales.

Que es la observabilidad

La observabilidad usa logs, metricas y trazas para comprender el estado interno del sistema. Mejora la operacion porque permite detectar incidentes y responder con informacion concreta.

Archivos principales entregados: README.md, docs/Resumen_Tecnico_DevOps.docx, docs/resumen-tecnico.md, docs/reflexion-critica.md, login.html, tests/test_project.py y .github/workflows/ci.yml.